POE Part 1

CLDV6211

Student Number: ST10493664

Name: Reabetswe Tebogo Fafudi

Module Code: CLDV6211

Class Group: Group 4

Table of Contents

Project Description 2
Part 1A – Design the EventEase Database 3
ERD – Entity Relationship Diagram: 3
Database Scripts 3
Bookings Table: 3
Events Table: 4
Venues Table: 4
Part 1B – Develop the Web Application 6
Screenshots 6
Solution Explorer 6
SQL Server Object Explorer 7
Packages 7
Models 8
Views 9
Controllers 10
Part 1C – Local Persistence Setup 12
Screenshot 12
Part 1D – Cloud Computing Basics 13
References 14

Project Description

GitHub Repo Link (Main): https://github.com/reabetswef/ST10493664-CLDV6211-POE.git

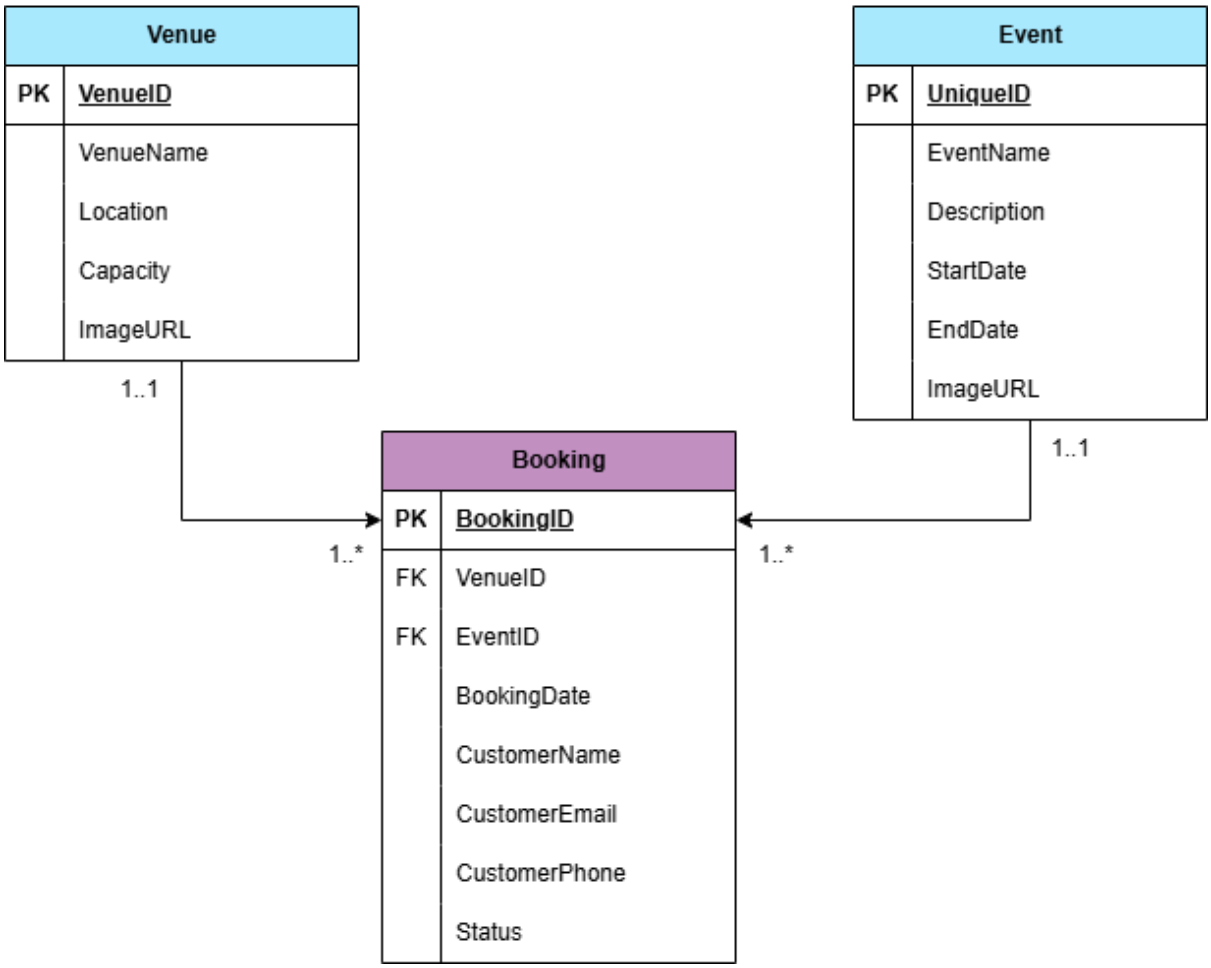
Commits & Completed project were submitted to my own repo before the link below was provided by the organisation

GitHub Repo Link (Organisation): https://github.com/EMGPPT/cldv6211-g4-2026-part1-reabetswef.git

YouTube Video Link: https://youtu.be/VKkIo6Vn8Vs

Part 1A – Design the EventEase Database

ERD – Entity Relationship Diagram:



Database Scripts

Bookings Table:

```
CREATE TABLE [dbo].[Bookings] (
    [BookingId]     INT          IDENTITY (1, 1) NOT NULL,
    [VenueId]       INT          NOT NULL,
    [EventId]       INT          NOT NULL,
    [BookingDate]   DATETIME2 (7)  NOT NULL,
    [CustomerName]  NVARCHAR (100) NOT NULL,
    [CustomerEmail] NVARCHAR (100) NULL,
    [CustomerPhone] NVARCHAR (20)  NULL,
    [Status]        INT          NOT NULL,
    CONSTRAINT [PK_Bookings] PRIMARY KEY CLUSTERED ([BookingId] ASC),
```

```
    CONSTRAINT [FK_Bookings_Events_EventId] FOREIGN KEY ([EventId]) REFERENCES [dbo].[Events]
([EventId]),
    CONSTRAINT [FK_Bookings_Venues_VenueId] FOREIGN KEY ([VenueId]) REFERENCES
[dbo].[Venues] ([VenueId])
);


GO
CREATE NONCLUSTERED INDEX [IX_Bookings_EventId]
    ON [dbo].[Bookings]([EventId] ASC);


GO
CREATE NONCLUSTERED INDEX [IX_Bookings_VenueId_BookingDate]
    ON [dbo].[Bookings]([VenueId] ASC, [BookingDate] ASC);
```

Events Table:

```
CREATE TABLE [dbo].[Events] (
    [EventId]     INT           IDENTITY (1, 1) NOT NULL,
    [EventName]   NVARCHAR (100) NOT NULL,
    [Description] NVARCHAR (500) NOT NULL,
    [StartDate]   DATETIME2 (7)  NOT NULL,
    [EndDate]     DATETIME2 (7)  NOT NULL,
    [ImageUrl]    NVARCHAR (MAX) NOT NULL,
    CONSTRAINT [PK_Events] PRIMARY KEY CLUSTERED ([EventId] ASC)
);
```

Venues Table:

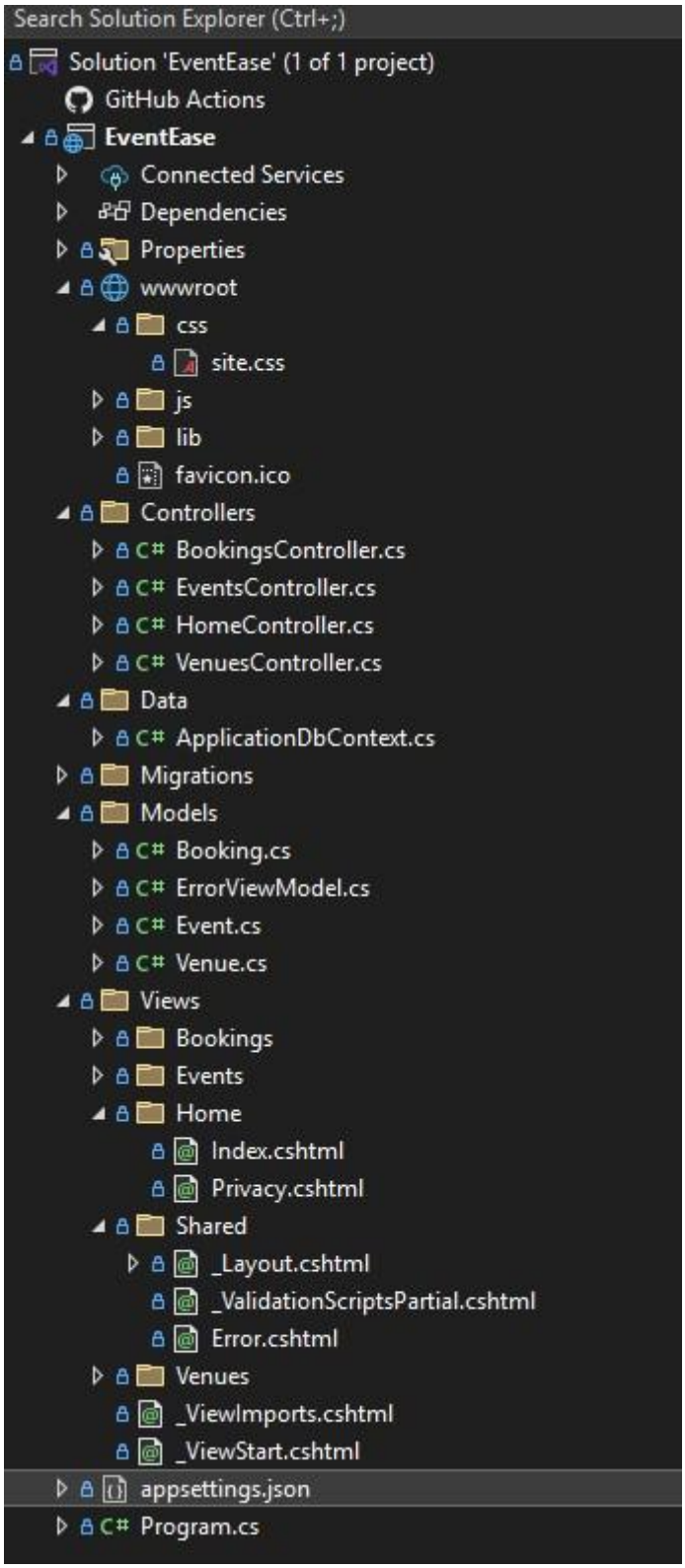
```
CREATE TABLE [dbo].[Venues] (
    [VenueId]   INT           IDENTITY (1, 1) NOT NULL,
    [VenueName] NVARCHAR (100) NOT NULL,
```

```
    [Location]  NVARCHAR (200) NOT NULL,
    [Capacity]  INT            NOT NULL,
    [ImageUrl]  NVARCHAR (MAX) NOT NULL,
    CONSTRAINT [PK_Venues] PRIMARY KEY CLUSTERED ([VenueId] ASC)
);
```

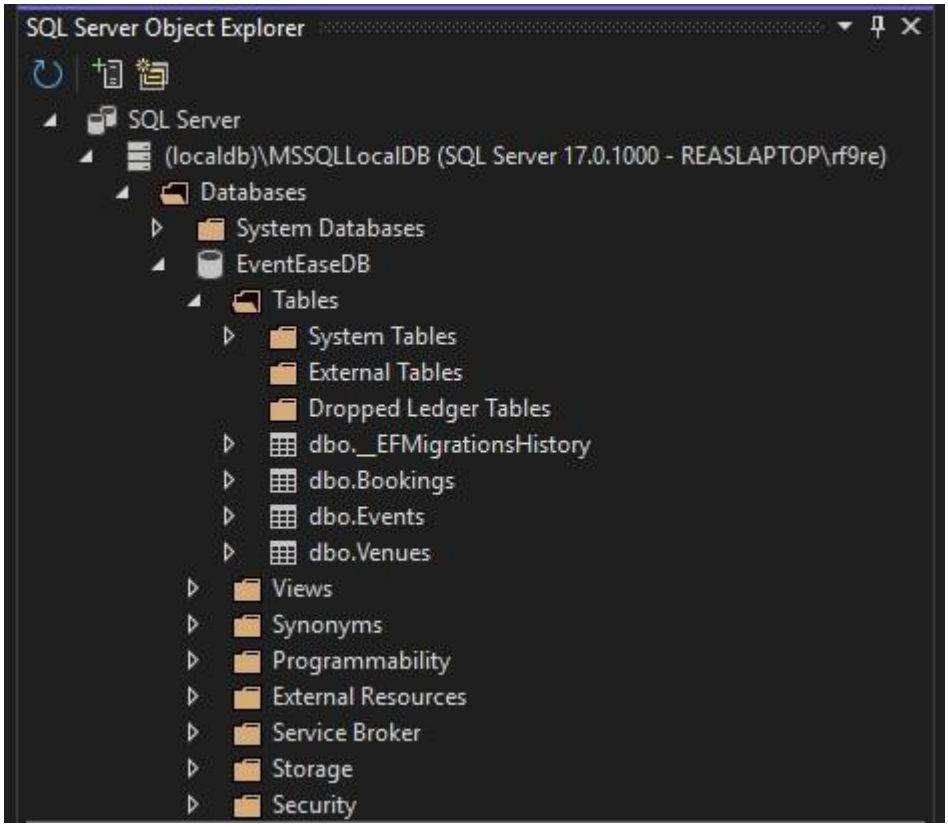
Part 1B – Develop the Web Application

Screenshots

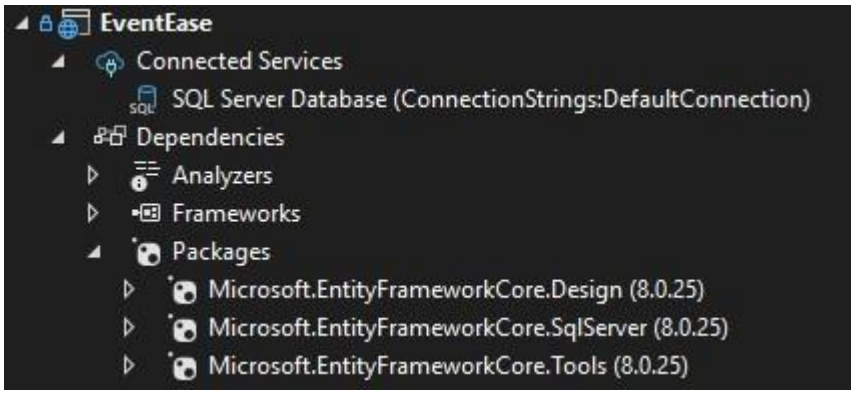
Solution Explorer



SQL Server Object Explorer



Packages



Models

Booking.cs

```
using System.ComponentModel.DataAnnotations;
using System.ComponentModel.DataAnnotations.Schema;

namespace EventEase.Models
{
    public class Booking
    {
        [Key]
        public int BookingId { get; set; }

        [Required]
        [Display(Name = "Venue")]
        public int VenueId { get; set; }

        [Required]
        [Display(Name = "Event")]
        public int EventId { get; set; }

        [Required(ErrorMessage = "Booking date is required")]
        [Display(Name = "Booking Date")]
        [DataType(DataType.DateTime)]
        public DateTime BookingDate { get; set; }

        [Required(ErrorMessage = "Customer name is required")]
        [StringLength(100, ErrorMessage = "Customer name cannot exceed 100 characters")]
        [Display(Name = "Customer Name")]
        public string CustomerName { get; set; } = string.Empty;

        [EmailAddress(ErrorMessage = "Invalid email address")]
        [StringLength(100)]
        [Display(Name = "Customer Email")]
        public string? CustomerEmail { get; set; }
```

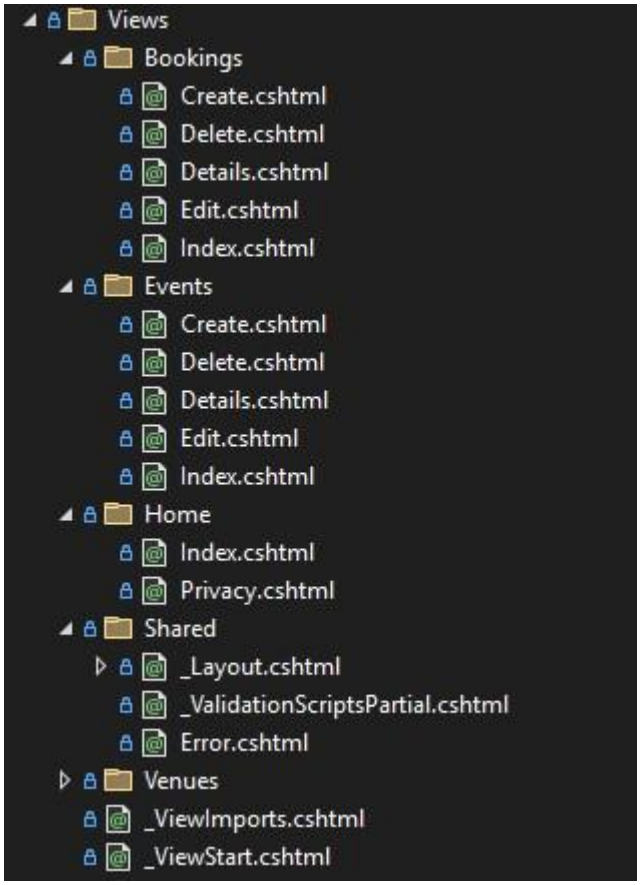
Event.cs

```
using System.ComponentModel.DataAnnotations;
using System.ComponentModel.DataAnnotations.Schema;

namespace EventEase.Models
{
    public class Event
    {
        [Key]
        public int EventId { get; set; }

        [Required(ErrorMessage = "Event name is required")]
        [StringLength(100, ErrorMessage = "Event name cannot exceed 100 characters")]
        [Display(Name = "Event Name")]
        public string EventName { get; set; } = string.Empty;

        [Required(ErrorMessage = "Description is required")]
        [StringLength(500, ErrorMessage = "Description cannot exceed 500 characters")]
        [DataType(DataType.MultilineText)]
        public string Description { get; set; } = string.Empty;

        [Required(ErrorMessage = "Start date is required")]
        [Display(Name = "Start Date")]
        [DataType(DataType.DateTime)]
        [DisplayFormat(DataFormatString = "{0:yyyy-MM-ddTHH:mm}", ApplyFormatInEditMode = true)]
        public DateTime StartDate { get; set; }

        [Required(ErrorMessage = "End date is required")]
        [Display(Name = "End Date")]
        [DataType(DataType.DateTime)]
        [DisplayFormat(DataFormatString = "{0:yyyy-MM-ddTHH:mm}", ApplyFormatInEditMode = true)]
        public DateTime EndDate { get; set; }

        [Display(Name = "Image URL")]
```

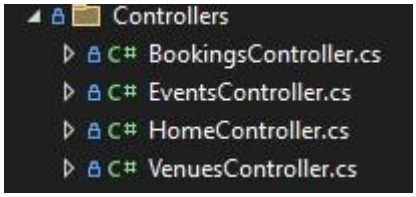
Venue.cs

```
using System.ComponentModel.DataAnnotations;
using System.ComponentModel.DataAnnotations.Schema;

namespace EventEase.Models
{
    public class Venue
    {
        [Key]
        public int VenueId { get; set; }

        [Required(ErrorMessage = "Venue name is required")]
        [StringLength(100, ErrorMessage = "Venue name cannot exceed 100 characters")]
        [Display(Name = "Venue Name")]
        public string VenueName { get; set; } = string.Empty;

        [Required(ErrorMessage = "Location is required")]
        [StringLength(200, ErrorMessage = "Location cannot exceed 200 characters")]
        public string Location { get; set; } = string.Empty;

        [Required(ErrorMessage = "Capacity is required")]
        [Range(1, 10000, ErrorMessage = "Capacity must be between 1 and 10,000")]
        public int Capacity { get; set; }

        [Display(Name = "Image URL")]
        [DataType(DataType.ImageUrl)]
        public string ImageUrl { get; set; } = string.Empty;

        // Navigation property - one venue can have many bookings
        public virtual ICollection<Booking>? Bookings { get; set; }
    }
}
```

Views


Views
Bookings
Create.cshtml
Delete.cshtml
Details.cshtml
Edit.cshtml
Index.cshtml
Events
Create.cshtml
Delete.cshtml
Details.cshtml
Edit.cshtml
Index.cshtml
Home
Index.cshtml
Privacy.cshtml
Shared
_Layout.cshtml
_ValidationScriptsPartial.cshtml
Error.cshtml
Venues
_ViewImports.cshtml
_ViewStart.cshtml

Controllers



BookingsController.cs

EventEase | EventEase.Controllers.BookingsController | _context

```
1   using Microsoft.AspNetCore.Mvc;
2   using Microsoft.EntityFrameworkCore;
3   using EventEase.Data;
4   using EventEase.Models;
5
6   namespace EventEase.Controllers
7   {
        1 reference
8       public class BookingsController : Controller
9       {
10          private readonly ApplicationDbContext _context;
11
            0 references
12          public BookingsController(ApplicationDbContext context)
13          {
14              _context = context;
15          }
16
17          // GET: Bookings
            3 references
18          public async Task<IActionResult> Index()
19          {
20              var bookings = _context.Bookings
21                  .Include(b => b.Venue)
22                  .Include(b => b.Event);
23              return View(await bookings.ToListAsync());
24          }
25
26          // GET: Bookings/Details/5
            0 references
27          public async Task<IActionResult> Details(int? id)
28          {
29              if (id == null)
30              {
31                  return NotFound();
32              }
33
34              var booking = await _context.Bookings
35                  .Include(b => b.Venue)
```

100 % | No issues found | Ln: 1 | Ch: 1 | SPC | CRLF

EventsController.cs

EventEase | EventEase.Controllers.EventsController | _context

```
using Microsoft.AspNetCore.Mvc;
using Microsoft.EntityFrameworkCore;
using EventEase.Data;
using EventEase.Models;

namespace EventEase.Controllers
{
    1 reference
    public class EventsController : Controller
    {
        private readonly ApplicationDbContext _context;

        0 references
        public EventsController(ApplicationDbContext context)
        {
            _context = context;
        }

        // GET: Events
        4 references
        public async Task<IActionResult> Index()
        {
            return View(await _context.Events.ToListAsync());
        }

        // GET: Events/Details/5
        0 references
        public async Task<IActionResult> Details(int? id)
        {
            if (id == null)
            {
                return NotFound();
            }

            var @event = await _context.Events
                .FirstOrDefaultAsync(m => m.EventId == id);

            if (@event == null)
            {
```

100 % | No issues found | Ln: 1 | Ch: 1 | SPC | CRLF

VenuesController.cs

EventEase | EventEase.Controllers.VenuesController | _context

```
using Microsoft.AspNetCore.Mvc;
using Microsoft.EntityFrameworkCore;
using EventEase.Data;
using EventEase.Models;

namespace EventEase.Controllers
{
    1 reference
    public class VenuesController : Controller
    {
        private readonly ApplicationDbContext _context;

        0 references
        public VenuesController(ApplicationDbContext context)
        {
            _context = context;
        }

        // GET: Venues
        4 references
        public async Task<IActionResult> Index()
        {
            return View(await _context.Venues.ToListAsync());
        }

        // GET: Venues/Details/5
        0 references
        public async Task<IActionResult> Details(int? id)
        {
            if (id == null)
            {
                return NotFound();
            }

            var venue = await _context.Venues
                .FirstOrDefaultAsync(m => m.VenueId == id);

            if (venue == null)
            {
```

100 % | No issues found | Ln: 1 | Ch: 1 | SPC | CRLF

Part 1C – Local Persistence Setup

Screenshot

appsettings.json

Schema: https://www.schemastore.org/appsettings.json

```
1  {
2      "Logging": {
3          "LogLevel": {
4              "Default": "Information",
5              "Microsoft.AspNetCore": "Warning"
6          }
7      },
8      "AllowedHosts": "*",
9      "ConnectionStrings": {
10         "DefaultConnection": "Server=(localdb)\\mssqllocaldb;Database=EventEaseDB;Trusted_Connection=True;MultipleActiveResultSets=true;TrustServerCertificate=True"
11     }
12 }
```

Part 1D – Cloud Computing Basics

1. Differences Between Cloud and On-Premises Deployment

Deploying an application in the cloud differs significantly from on-premises deployment in several aspects, notably security, deployment speed, and resource management. In terms of security, cloud providers such as Amazon Web Services (AWS) or Microsoft Azure offer built-in security features including encryption at rest and in transit, automated security updates, and robust identity management (Rountree & Castrillo, 2019). In contrast, on-premises systems require organizations to implement and maintain security measures themselves, which can be costly and prone to human error. Regarding deployment speed, cloud environments enable rapid provisioning of servers and resources, allowing developers to deploy applications in minutes rather than days or weeks, as is often the case with on-premises setups where hardware procurement and configuration delays are common (Armbrust et al., 2010). For resource management, cloud computing provides elasticity, automatically scaling resources up or down according to demand, which is particularly useful for applications with fluctuating workloads. For example, a ticketing application like EventEase could dynamically scale server capacity during high-demand periods, such as when popular event tickets go on sale, without manual intervention. On-premises environments lack this flexibility, typically requiring over-provisioning to handle peak loads.

2. Differences Between IaaS, PaaS, and SaaS

Cloud services are typically categorized into Infrastructure as a Service (IaaS), Platform as a Service (PaaS), and Software as a Service (SaaS), each offering different levels of abstraction. IaaS provides fundamental computing resources such as virtual machines, storage, and networking, giving organizations control over operating systems and applications while still outsourcing hardware management (Mell & Grance, 2011). PaaS offers a higher-level environment, including operating systems, databases, and development frameworks, allowing developers to build and deploy applications without managing the underlying infrastructure (Buyya et al., 2013). SaaS delivers fully functional software applications over the internet, such as Google Workspace or Salesforce, which users can access directly without handling any backend infrastructure. For EventEase, PaaS would likely be the most beneficial choice when moving to the cloud because it provides a ready-to-use development environment that accelerates application deployment, reduces operational overhead, and simplifies scaling during peak event periods. Unlike IaaS, PaaS abstracts infrastructure management, and unlike SaaS, it allows for custom application development, which is crucial for tailoring EventEase to its unique ticketing and event management requirements.

References

Armbrust, M., Fox, A., Griffith, R., Joseph, A. D., Katz, R., Konwinski, A., Lee, G., Patterson, D., Rabkin, A., Stoica, I. & Zaharia, M., 2010. *A view of cloud computing*. Communications of the ACM, 53(4), pp.50–58.

Buyya, R., Broberg, J. & Goscinski, A., 2013. *Cloud Computing: Principles and Paradigms*. Hoboken, NJ: Wiley.

Mell, P. & Grance, T., 2011. *The NIST definition of cloud computing*. Gaithersburg, MD: National Institute of Standards and Technology.

Rountree, N. & Castrillo, G., 2019. *The Basics of Cloud Computing*. 2nd ed. Boston: Syngress.